

```

print("Dataset Information:")
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Rows:")
print(df.duplicated().sum())

print("\nDataset Shape:")
print(df.shape)

```

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 300 entries, 0 to 299

Data columns (total 16 columns):

#	Column	Non-Null Count	Dtype
0	Accident ID	300 non-null	object
1	Date	300 non-null	object
2	District	300 non-null	object
3	Location type	300 non-null	object
4	Road type	300 non-null	object
5	Weather condition	300 non-null	object
6	Accident time	300 non-null	object
7	Vehicle type	300 non-null	object
8	Number of vehicles involved	300 non-null	int64
9	Fatalities	300 non-null	int64
10	Injuries	300 non-null	int64
11	Accident severity	300 non-null	object
12	Speed limit	300 non-null	int64
13	Road condition	300 non-null	object
14	Traffic level	300 non-null	object
15	Main cause of accident	300 non-null	object

dtypes: int64(4), object(12)

memory usage: 37.6+ KB

None

Missing Values:

Accident ID	0
Date	0
District	0
Location type	0
Road type	0
Weather condition	0
Accident time	0
Vehicle type	0
Number of vehicles involved	0
Fatalities	0
Injuries	0
Accident severity	0
Speed limit	0
Road condition	0
Traffic level	0
Main cause of accident	0

dtype: int64

Duplicate Rows:

0

Dataset Shape:

(300, 16)

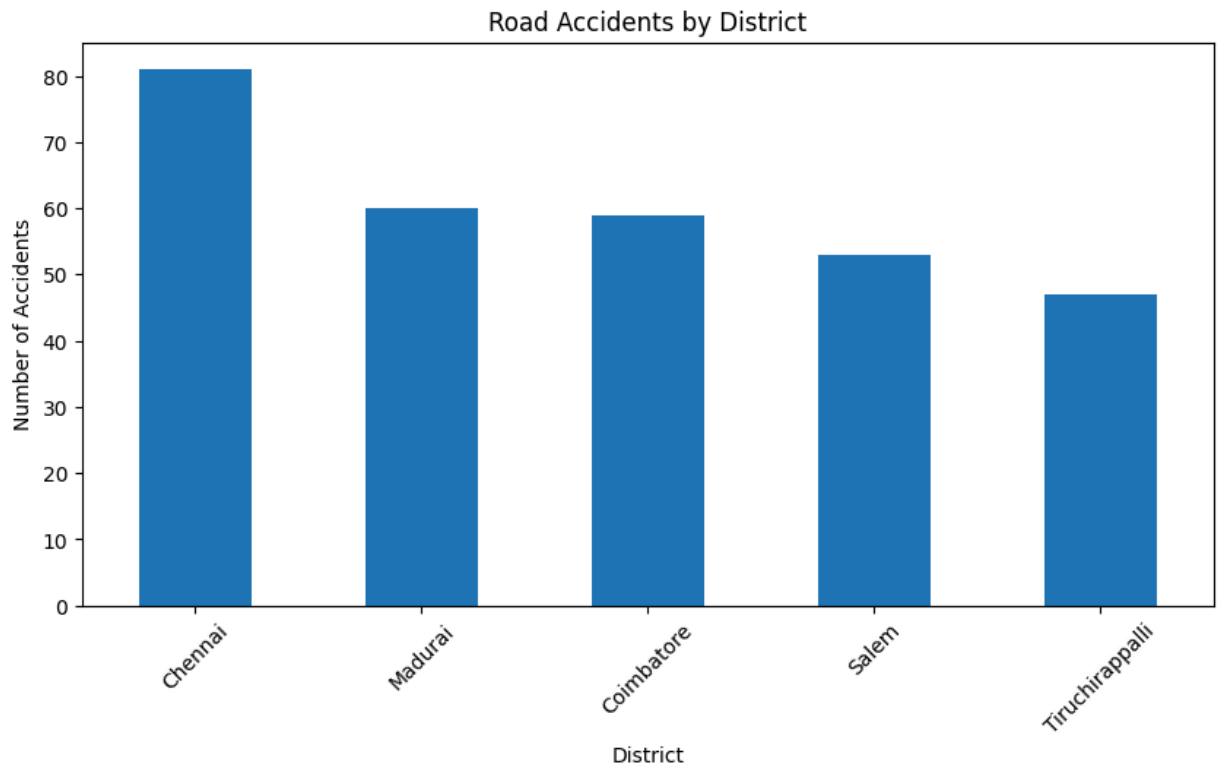
```
district_counts = df['District'].value_counts()
```

```

plt.figure(figsize=(10,5))
district_counts.plot(kind='bar')
plt.title('Road Accidents by District')
plt.xlabel('District')

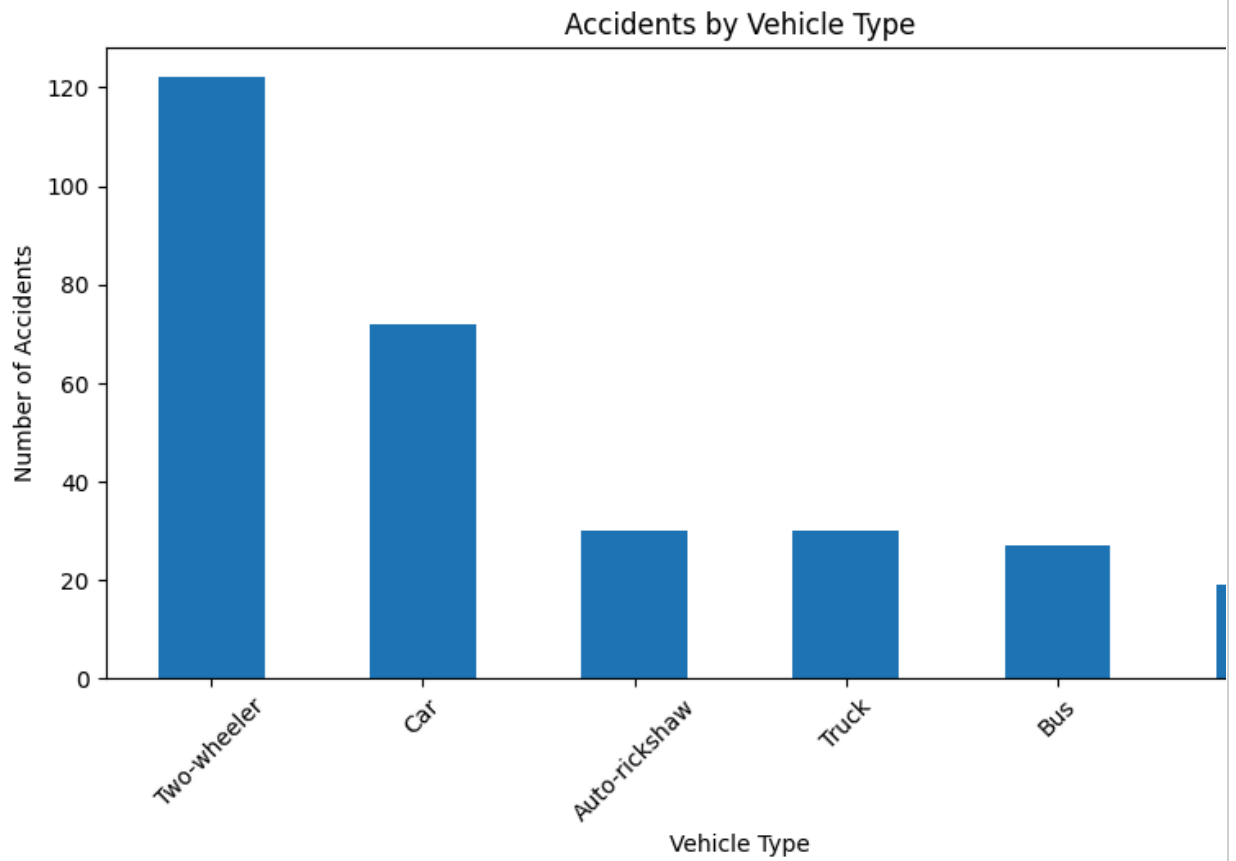
```

```
plt.ylabel('Number of Accidents')  
plt.xticks(rotation=45)  
plt.show()
```



```
vehicle_counts = df['Vehicle type'].value_counts()
```

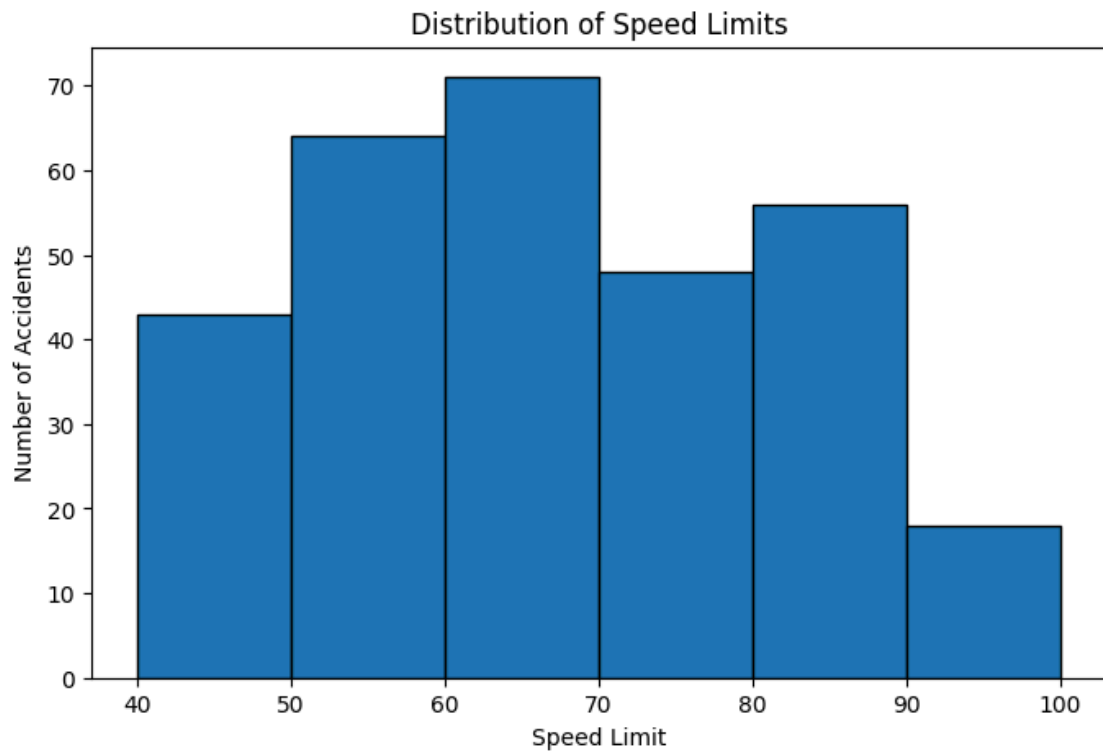
```
plt.figure(figsize=(10,5))  
vehicle_counts.plot(kind='bar')  
plt.title('Accidents by Vehicle Type')  
plt.xlabel('Vehicle Type')  
plt.ylabel('Number of Accidents')  
plt.xticks(rotation=45)  
plt.show()
```



```
plt.figure(figsize=(8,5))

plt.hist(
    df['Speed limit'],
    bins=6,
    edgecolor='black'
)

plt.title('Distribution of Speed Limits')
plt.xlabel('Speed Limit')
plt.ylabel('Number of Accidents')
plt.show()
```

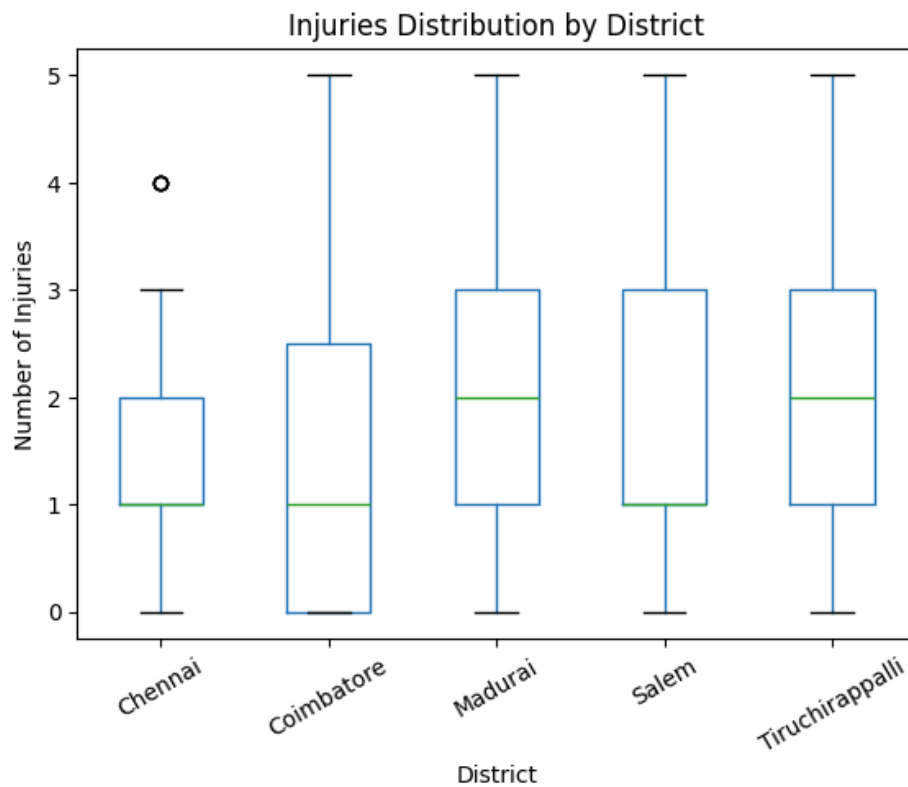


```
plt.figure(figsize=(10,6))

df.boxplot(
    column='Injuries',
    by='District',
    grid=False
)

plt.title('Injuries Distribution by District')
plt.suptitle('')
plt.xlabel('District')
plt.ylabel('Number of Injuries')
plt.xticks(rotation=30)
plt.show()
```

<Figure size 1000x600 with 0 Axes>

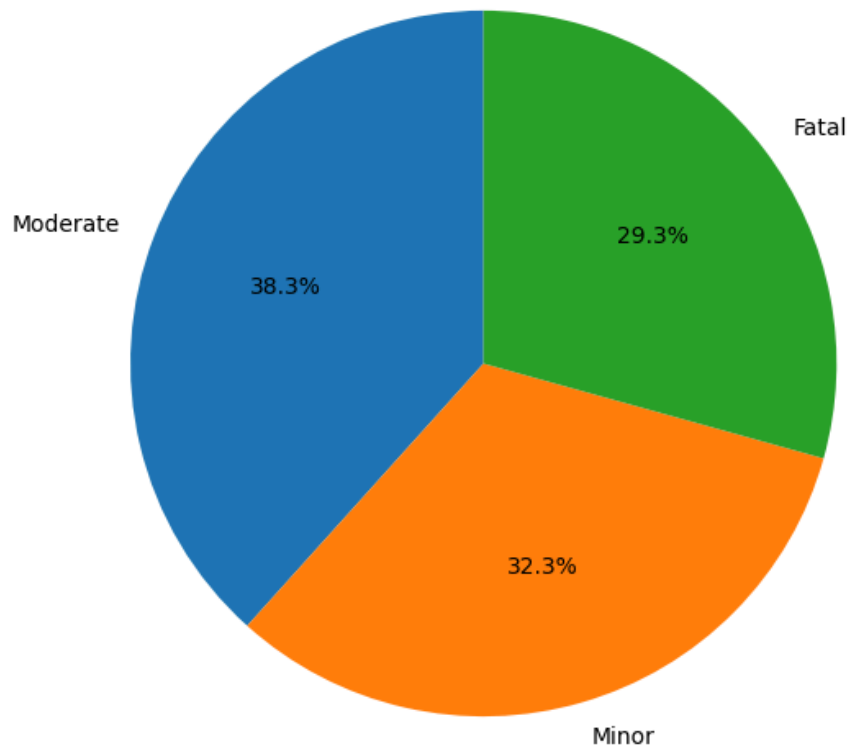


```
plt.figure(figsize=(7,7))

df['Accident severity'].value_counts().plot(
    kind='pie',
    autopct='%1.1f%%',
    startangle=90
)

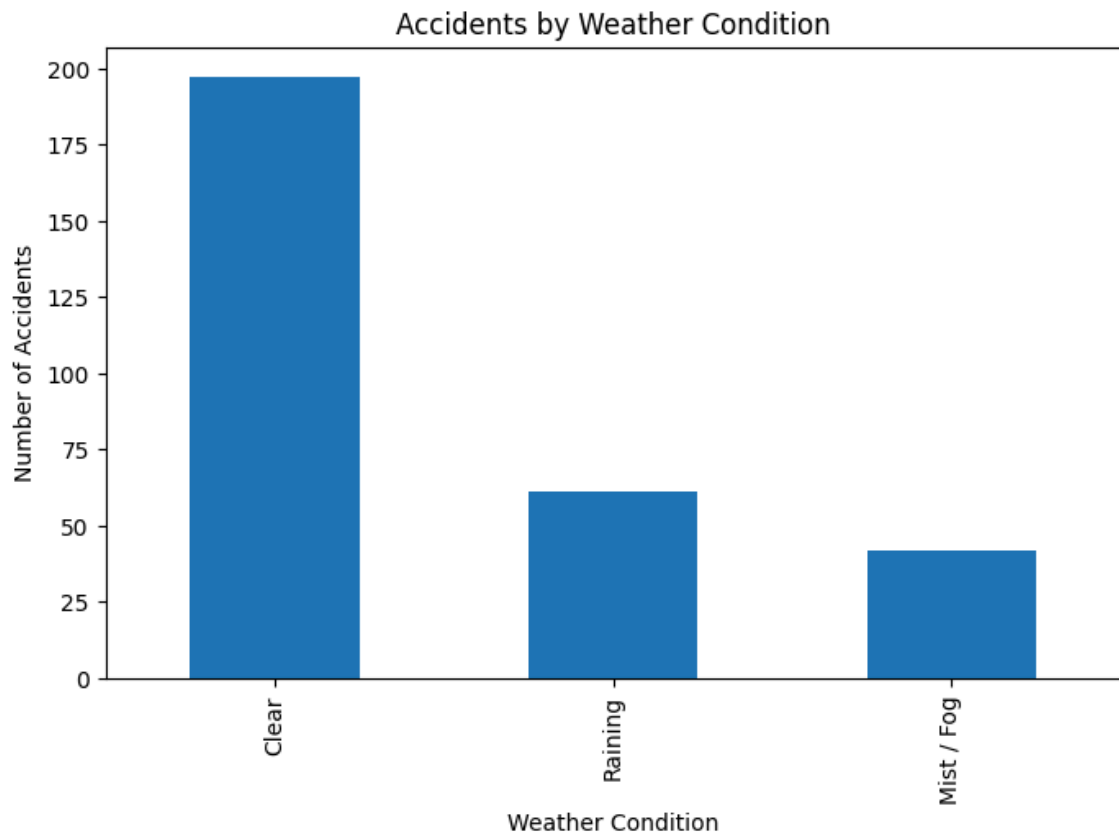
plt.title('Accident Severity Distribution')
plt.ylabel('')
plt.show()
```

Accident Severity Distribution



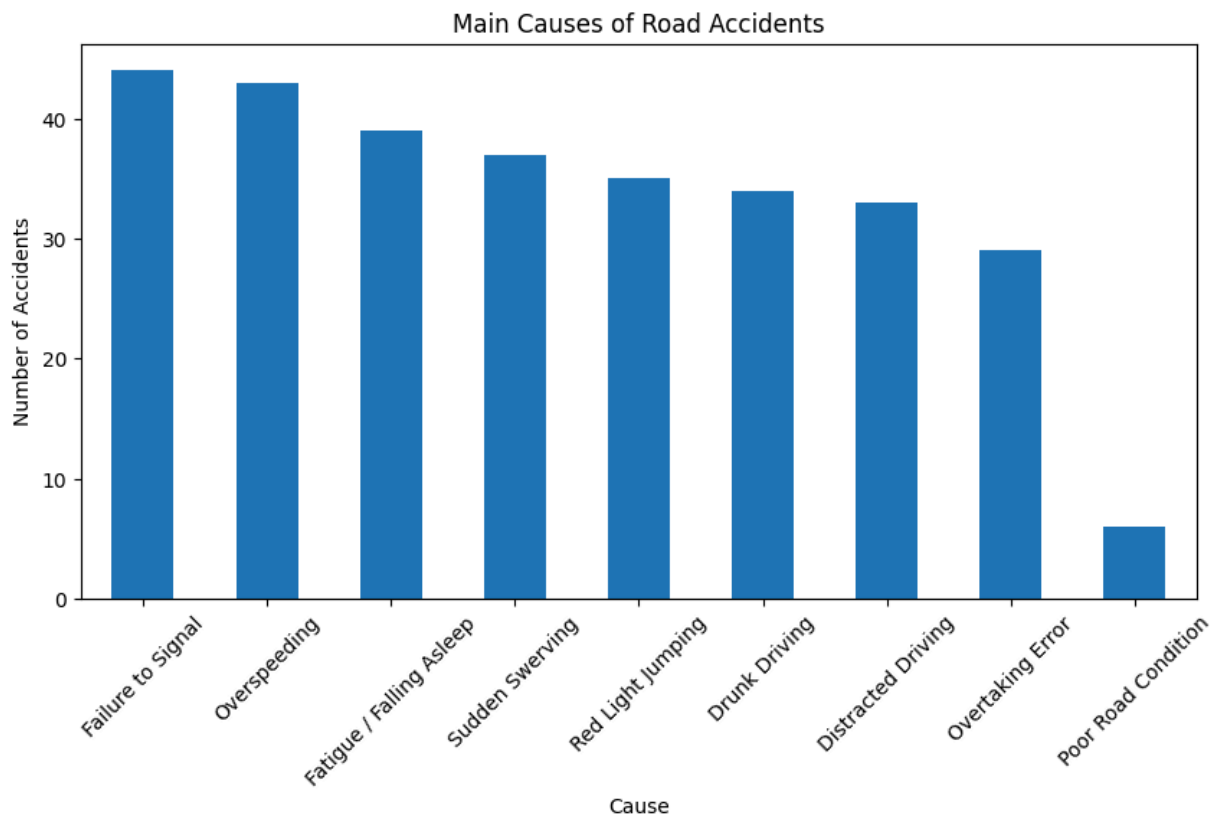
```
weather_counts = df['Weather condition'].value_counts()

plt.figure(figsize=(8,5))
weather_counts.plot(kind='bar')
plt.title('Accidents by Weather Condition')
plt.xlabel('Weather Condition')
plt.ylabel('Number of Accidents')
plt.show()
```



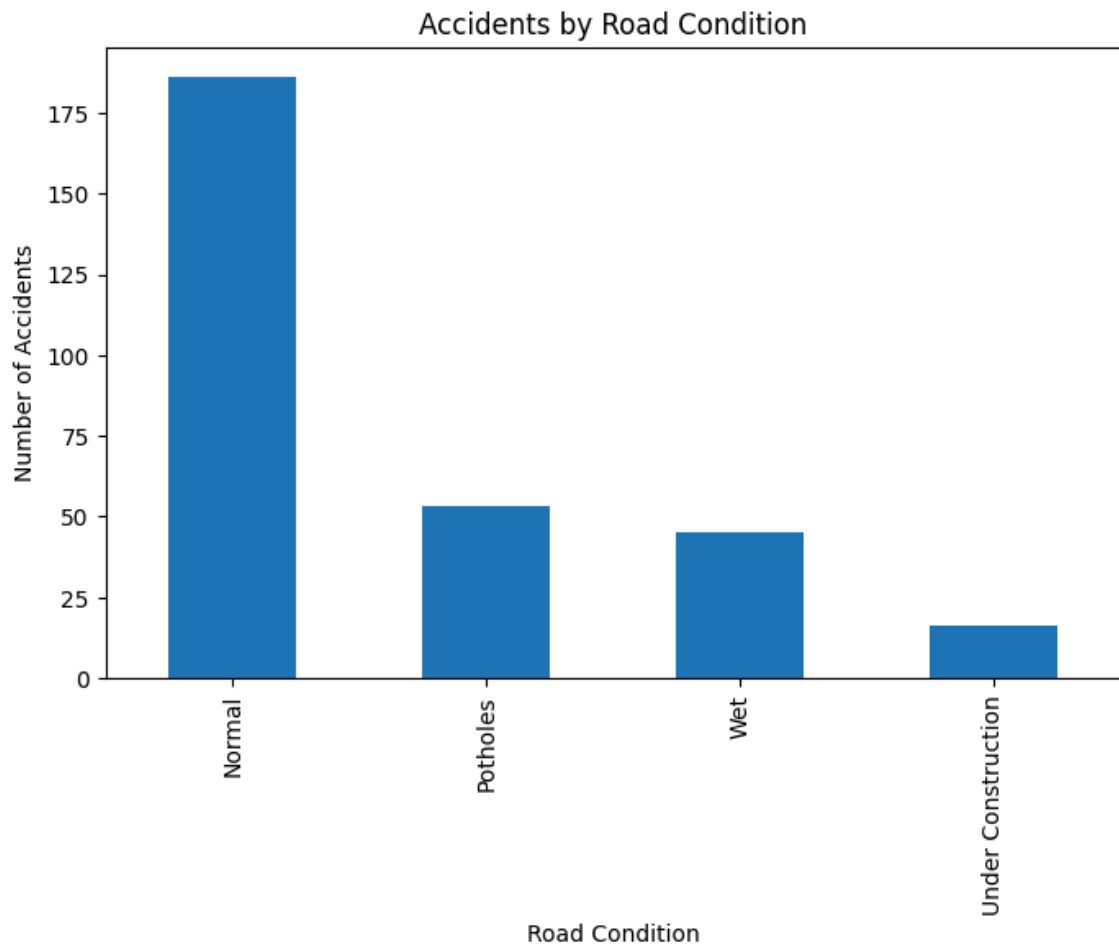
```
cause_counts = df['Main cause of accident'].value_counts()

plt.figure(figsize=(10,5))
cause_counts.plot(kind='bar')
plt.title('Main Causes of Road Accidents')
plt.xlabel('Cause')
plt.ylabel('Number of Accidents')
plt.xticks(rotation=45)
plt.show()
```



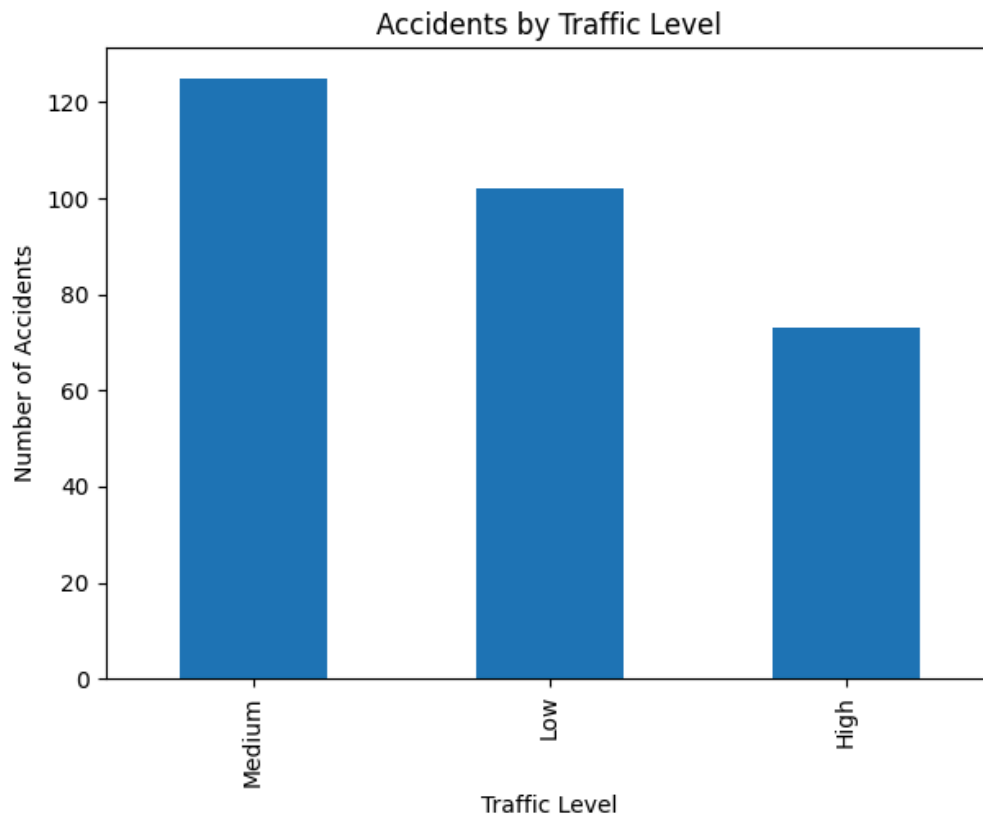
```
road_counts = df['Road condition'].value_counts()
```

```
plt.figure(figsize=(8,5))
road_counts.plot(kind='bar')
plt.title('Accidents by Road Condition')
plt.xlabel('Road Condition')
plt.ylabel('Number of Accidents')
plt.show()
```



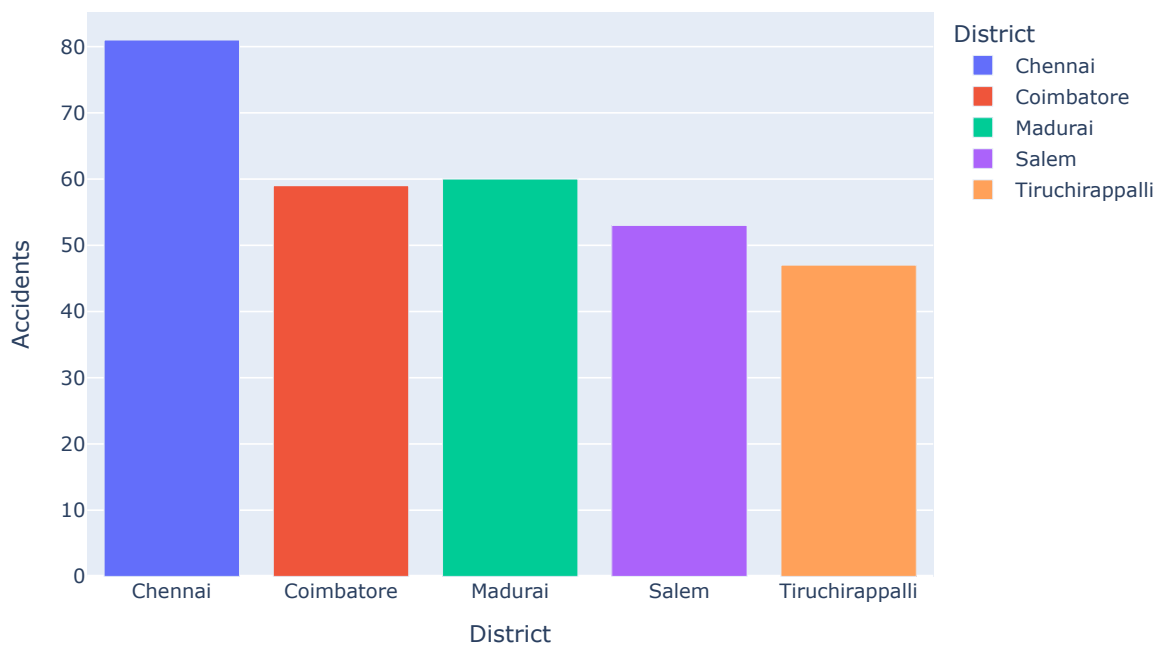
```
traffic_counts = df['Traffic level'].value_counts()
```

```
plt.figure(figsize=(7,5))  
traffic_counts.plot(kind='bar')  
plt.title('Accidents by Traffic Level')  
plt.xlabel('Traffic Level')  
plt.ylabel('Number of Accidents')  
plt.show()
```



```
district_data = df.groupby('District', as_index=False).agg(  
    Accidents=('Accident ID', 'count'),  
    Fatalities=('Fatalities', 'sum'),  
    Injuries=('Injuries', 'sum')  
)  
  
fig = px.bar(  
    district_data,  
    x='District',  
    y='Accidents',  
    color='District',  
    title='Road Accidents by District'  
)  
  
fig.show()
```

Road Accidents by District



```
df['Date'] = pd.to_datetime(df['Date'])

monthly_accidents = (
    df.groupby(df['Date'].dt.to_period('M'))
    .size()
)

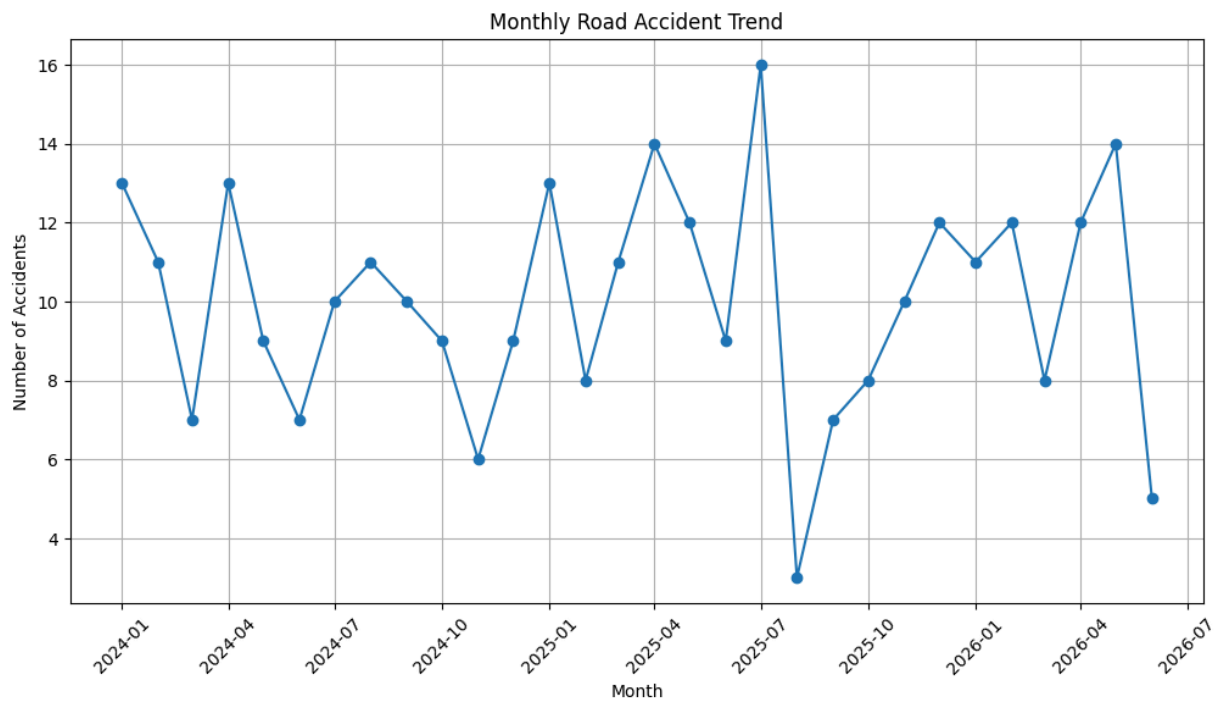
monthly_accidents.index = monthly_accidents.index.to_timestamp()

plt.figure(figsize=(12,6))

plt.plot(
    monthly_accidents.index,
    monthly_accidents.values,
    marker='o'
)

plt.title('Monthly Road Accident Trend')
plt.xlabel('Month')
plt.ylabel('Number of Accidents')
plt.xticks(rotation=45)
plt.grid(True)

plt.show()
```



```
import seaborn as sns

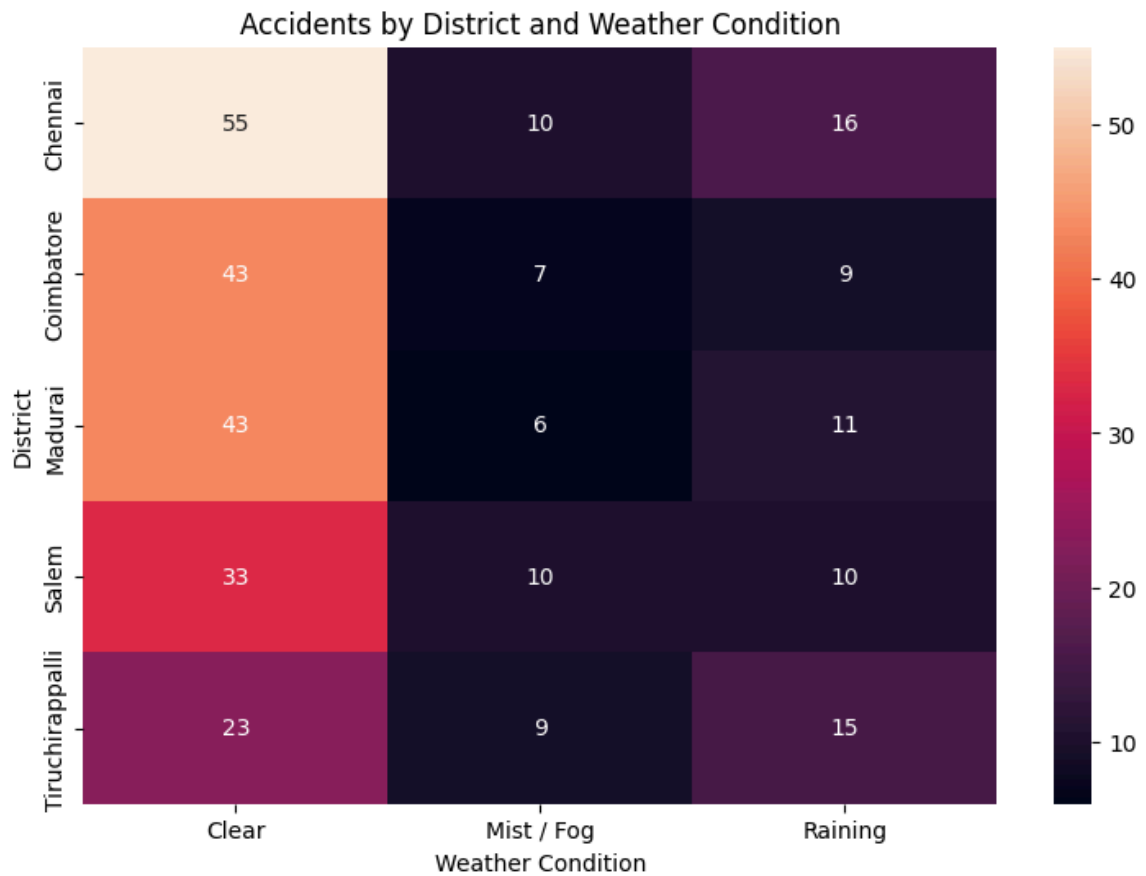
heatmap_data = pd.crosstab(
    df['District'],
    df['Weather condition']
)

plt.figure(figsize=(9,6))

sns.heatmap(
    heatmap_data,
    annot=True,
    fmt='d'
)

plt.title('Accidents by District and Weather Condition')
plt.xlabel('Weather Condition')
plt.ylabel('District')

plt.show()
```

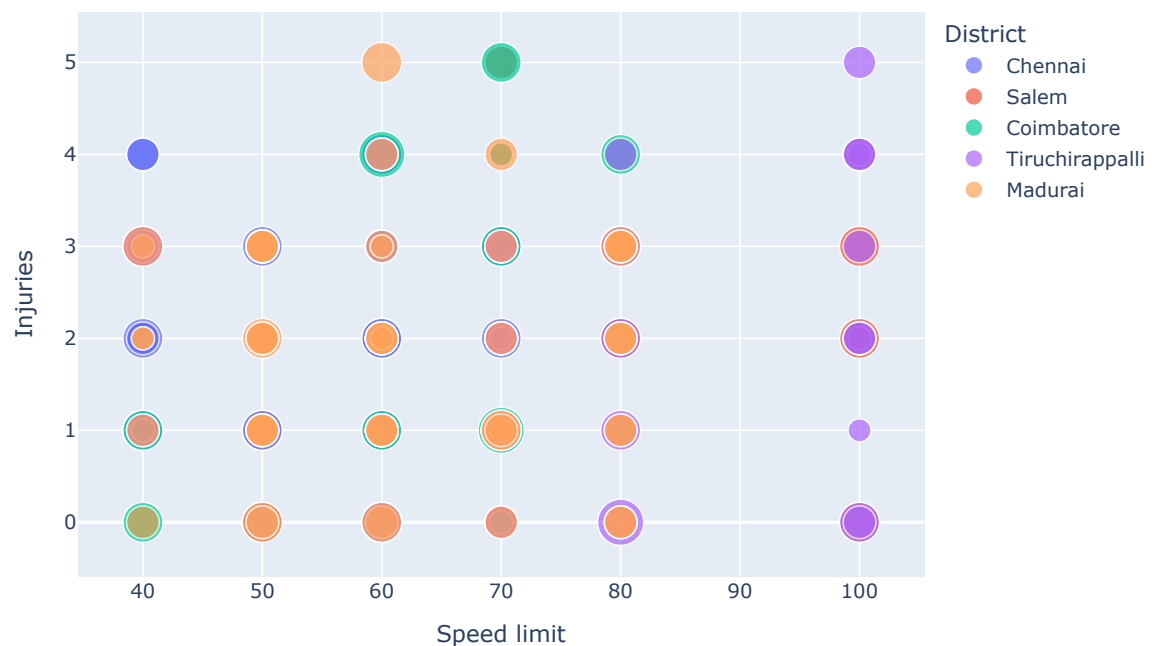


```
import plotly.express as px

fig = px.scatter(
    df,
    x='Speed limit',
    y='Injuries',
    color='District',
    size='Number of vehicles involved',
    hover_data=[
        'Accident severity',
        'Vehicle type',
        'Weather condition',
        'Main cause of accident'
    ],
    title='Interactive: Speed Limit vs Injuries'
)

fig.show()
```

Interactive: Speed Limit vs Injuries



```
import pandas as pd
import numpy as np

# Set random seed for reproducibility
np.random.seed(42)

# -----
# PART 1: GENERATE ORIGINAL 200 RECORDS
# -----

n_records = 200

districts = ['Chennai', 'Coimbatore', 'Madurai', 'Salem', 'Tiruchirappalli']
district_weights = [0.28, 0.24, 0.18, 0.16, 0.14]

location_types = ['Urban', 'Suburban', 'Rural']
road_types = ['National Highway', 'State Highway', 'City Road', 'District Road']
weather_conditions = ['Clear', 'Raining', 'Mist / Fog']
weather_weights = [0.70, 0.18, 0.12]

time_slots = [
    'Early Morning (00:00 - 06:00)',
    'Morning (06:00 - 12:00)',
    'Afternoon (12:00 - 18:00)',
    'Evening (18:00 - 22:00)',
    'Night (22:00 - 00:00)'
]

vehicle_types = ['Two-wheeler', 'Car', 'Auto-rickshaw', 'Bus', 'Truck', 'Lorry']
vehicle_weights = [0.42, 0.22, 0.12, 0.10, 0.08, 0.06]

causes = [
    'Overspeeding',
    'Drunk Driving',
    'Distracted Driving',
    'Red Light Jumping',
    'Sudden Swerving',
    'Fatigue / Falling Asleep'
```

```
'Overtaking Error',
'Failure to Signal'
]

road_conditions = ['Normal', 'Wet', 'Potholes', 'Under Construction']
traffic_levels = ['Low', 'Medium', 'High']

start_date = pd.to_datetime('2024-01-01')
end_date = pd.to_datetime('2026-06-30')

random_dates = start_date + pd.to_timedelta(
    np.random.randint(
        0,
        (end_date - start_date).days,
        n_records
    ),
    unit='D'
)

data = []

for i in range(n_records):

    acc_id = f"ACC{i+1:03d}"
    date = random_dates[i].strftime('%Y-%m-%d')

    district = np.random.choice(
        districts,
        p=district_weights
    )

    vehicle = np.random.choice(
        vehicle_types,
        p=vehicle_weights
    )

    weather = np.random.choice(
        weather_conditions,
        p=weather_weights
    )

    time_of_day = np.random.choice(time_slots)

    if district in ['Chennai', 'Coimbatore']:
        loc_type = np.random.choice(
            location_types,
            p=[0.55, 0.30, 0.15]
        )
    else:
        loc_type = np.random.choice(
            location_types,
            p=[0.25, 0.35, 0.40]
        )

    if loc_type == 'Rural':

        road_type = np.random.choice(
            ['National Highway', 'District Road', 'State Highway'],
            p=[0.45, 0.35, 0.20]
        )

        speed_limit = np.random.choice([50, 80])

    elif loc_type == 'Suburban':

        road_type = np.random.choice(
            ['National Highway', 'State Highway', 'District Road'],
```

```

        p=[0.40, 0.40, 0.20]
    )

    speed_limit = np.random.choice([60, 70, 80])

else:

    road_type = np.random.choice(
        ['City Road', 'State Highway', 'National Highway'],
        p=[0.50, 0.35, 0.15]
    )

    speed_limit = np.random.choice([40, 50, 60])

if weather == 'Raining':

    road_cond = np.random.choice(
        ['Wet', 'Normal'],
        p=[0.85, 0.15]
    )

else:

    road_cond = np.random.choice(
        ['Normal', 'Potholes', 'Under Construction'],
        p=[0.80, 0.15, 0.05]
    )

if (
    time_of_day in
    ['Morning (06:00 - 12:00)',
     'Evening (18:00 - 22:00)']
    and loc_type == 'Urban'
):
    traffic = 'High'

elif time_of_day in [
    'Early Morning (00:00 - 06:00)',
    'Night (22:00 - 00:00)'
]:
    traffic = 'Low'

else:
    traffic = 'Medium'

if time_of_day == 'Early Morning (00:00 - 06:00)':

    cause = np.random.choice(
        [
            'Fatigue / Falling Asleep',
            'Drunk Driving',
            'Overspeeding'
        ],
        p=[0.4, 0.4, 0.2]
    )

else:

    cause = np.random.choice(causes)

if (
    road_type == 'National Highway'
    or speed_limit >= 70
    or cause in [
        'Drunk Driving',
        'Fatigue / Falling Asleep'
    ]
):
    \.

```

```

severity = np.random.choice(
    ['Fatal', 'Moderate', 'Minor'],
    p=[0.45, 0.35, 0.20]
)

else:

    severity = np.random.choice(
        ['Minor', 'Moderate', 'Fatal'],
        p=[0.50, 0.40, 0.10]
    )

if severity == 'Fatal':

    fatalities = np.random.choice(
        [1, 2],
        p=[0.85, 0.15]
    )

    injuries = np.random.randint(0, 4)

elif severity == 'Moderate':

    fatalities = 0
    injuries = np.random.randint(1, 4)

else:

    fatalities = 0
    injuries = np.random.choice([0, 1])

vehicles_involved = (
    1
    if vehicle == 'Two-wheeler'
    and np.random.rand() > 0.6
    else np.random.choice([2, 3], p=[0.85, 0.15])
)

data.append({
    'Accident ID': acc_id,
    'Date': date,
    'District': district,
    'Location type': loc_type,
    'Road type': road_type,
    'Weather condition': weather,
    'Accident time': time_of_day,
    'Vehicle type': vehicle,
    'Number of vehicles involved': vehicles_involved,
    'Fatalities': fatalities,
    'Injuries': injuries,
    'Accident severity': severity,
    'Speed limit': speed_limit,
    'Road condition': road_cond,
    'Traffic level': traffic,
    'Main cause of accident': cause
})

```